

Documentação OpenAPI Gerenciador de Alunos

API REST para gestão acadêmica

Papel: Pessoa 6 - Documentação OpenAPI
Projeto acadêmico desenvolvido com NestJS e React/Vite
Data: 01/07/2026

Base URL local: <http://localhost:3000>
Swagger UI: <http://localhost:3000/api-docs>

Sumário

1. Introdução	3
2. Visão geral da API	3
3. Swagger UI e OpenAPI	3
4. Padrão de versionamento	4
5. Padrão de respostas de erro	4
6. Status codes	5
7. Endpoints	5
7.1 Health	5
7.2 Cursos	5
7.3 Alunos	7
7.4 Disciplinas	8
7.5 Matrículas	10
8. Observações sobre rotas de relacionamento	11
9. Fluxo de uso da API	11
10. Conclusão	11

1. Introdução

A API do projeto Gerenciador de Alunos foi desenvolvida para centralizar operações acadêmicas relacionadas a cursos, alunos, disciplinas e matrículas. O objetivo principal é disponibilizar uma interface REST clara, padronizada e testável, permitindo que o front-end em React/Vite consuma os dados do backend implementado em NestJS.

A documentação OpenAPI tem papel essencial no trabalho em equipe porque descreve, de forma formal e compreensível, os contratos de comunicação entre as partes do sistema. Em um projeto dividido entre oito papéis, essa documentação reduz ambiguidades, organiza responsabilidades e facilita a validação do comportamento esperado da API.

Com a especificação OpenAPI, os integrantes responsáveis pelo backend, front-end, testes, relacionamentos e apresentação conseguem consultar endpoints, parâmetros, schemas, exemplos de request e response, status codes e casos de erro em uma fonte única. Assim, a documentação facilita testes manuais, integração com o front-end, manutenção futura e apresentação acadêmica do projeto.

2. Visão geral da API

Item	Descrição
Nome	Gerenciador de Alunos API
Versão	1.0.0
Base URL	http://localhost:3000
Formato de comunicação	JSON
Autenticação	Não implementada
Persistência	Memória da aplicação
Swagger UI	http://localhost:3000/api-docs
OpenAPI JSON	http://localhost:3000/api-docs-json

A persistência em memória indica que os dados criados durante a execução da aplicação não são mantidos após reiniciar o servidor. Essa característica atende ao contexto atual do trabalho acadêmico, mas pode ser substituída futuramente por banco de dados relacional ou não relacional sem alterar a finalidade principal dos contratos da API.

3. Swagger UI e OpenAPI

A API possui Swagger UI integrado ao backend em NestJS. Com a aplicação em execução, a documentação interativa fica disponível no endereço <http://localhost:3000/api-docs>, permitindo visualizar os endpoints, conferir os schemas e testar chamadas diretamente pelo navegador.

O contrato OpenAPI em formato JSON pode ser consultado em <http://localhost:3000/api-docs-json>. Esse endpoint exposto pela aplicação permite que ferramentas externas de testes, geradores de clientes HTTP ou validadores de contrato consumam a especificação da API.

A aplicação utiliza as bibliotecas `@nestjs/swagger`, `swagger-ui-express` e `js-yaml`. O backend tenta carregar preferencialmente o arquivo `docs/openapi.yaml` como fonte principal da documentação, mantendo a interface visual do Swagger alinhada com a documentação oficial e com o JSON exposto pela API.

Caso o arquivo `docs/openapi.yaml` não esteja disponível no ambiente de execução, a aplicação possui fallback para gerar um documento básico com `DocumentBuilder`. Essa abordagem aumenta a

segurança operacional da documentação, pois evita que o Swagger fique indisponível caso o arquivo principal não seja encontrado.

Essa estrutura facilita testes manuais, integração com front-end, conferência de contratos e apresentação acadêmica, porque transforma a especificação em uma interface navegável, padronizada e executável.

4. Padrão de versionamento

A documentação utiliza o padrão SemVer, também conhecido como versionamento semântico. Nesse formato, a versão é representada por três números no padrão MAJOR.MINOR.PATCH.

Nível	Uso
MAJOR	Utilizado para mudanças incompatíveis, como remoção de campos obrigatórios, alteração de rotas ou mudanças que quebrem integrações existentes.
MINOR	Utilizado para novos recursos retrocompatíveis, como novos endpoints, novos filtros opcionais ou novos campos que não removem compatibilidade.
PATCH	Utilizado para correções de documentação, ajustes de texto, exemplos, descrição de schemas ou pequenas correções sem alteração de contrato.

O backend atual não utiliza prefixo /v1 nas rotas. Por esse motivo, a versão da API está registrada na especificação OpenAPI como 1.0.0, funcionando como referência formal do contrato publicado.

5. Padrão de respostas de erro

As respostas de erro seguem uma estrutura padronizada para facilitar a leitura, o tratamento no front-end e a depuração durante os testes. Um exemplo de erro de validação é apresentado abaixo.

```
{
  "statusCode": 400,
  "message": [
    "nome must be longer than or equal to 3 characters"
  ],
  "error": "Bad Request",
  "timestamp": "2026-07-01T12:00:00.000Z",
  "path": "/cursos"
}
```

Campo	Descrição
statusCode	Código HTTP associado ao erro retornado pela API.
message	Mensagem ou lista de mensagens explicando a causa do erro, geralmente relacionada a validações ou regras de negócio.
error	Descrição resumida do tipo de erro HTTP.
timestamp	Data e hora em que o erro ocorreu, no formato ISO 8601.
path	Rota chamada pelo cliente no momento em que o erro foi gerado.

6. Status codes

Status code	Significado
200 OK	Consulta, atualização ou exclusão realizada com sucesso.
201 Created	Recurso criado com sucesso.
400 Bad Request	Erro de validação ou regra de negócio.
404 Not Found	Recurso inexistente.
409 Conflict	Conflito de unicidade ou relacionamento.
500 Internal Server Error	Erro inesperado no servidor.

7. Endpoints

Os endpoints estão organizados por recurso para facilitar a consulta e a manutenção da documentação. Em todos os exemplos, considera-se a base URL local `http://localhost:3000`.

7.1 Health

Método	Rota	Descrição	Status
GET	/	Verifica a disponibilidade básica da aplicação.	200

Resposta esperada para a chamada GET /:

```
"Hello World!"
```

7.2 Cursos

Método	Rota	Descrição
POST	/cursos	Cria um novo curso.
GET	/cursos	Lista cursos com suporte a paginação e filtro por nome.
GET	/cursos/{id}	Consulta um curso específico pelo identificador.
PATCH	/cursos/{id}	Atualiza parcialmente os dados de um curso.
DELETE	/cursos/{id}	Remove um curso existente.

POST /cursos

Cria um novo curso no sistema.

```
{
  "nome": "Análise e Desenvolvimento de Sistemas",
  "descricao": "Curso superior de tecnologia focado em desenvolvimento de software."
}
```

Campo	Validacoes
nome	Obrigatorio, texto, entre 3 e 100 caracteres.
descrição	Obrigatorio, texto, entre 10 e 500 caracteres.

Status	Situação
201	Curso criado com sucesso.
400	Erro de validação dos campos enviados.

Status	Situação
409	Conflito de unicidade ou regra relacionada ao curso.

Exemplo de resposta 201:

```
{
  "id": 1,
  "nome": "Análise e Desenvolvimento de Sistemas",
  "descricao": "Curso superior de tecnologia focado em desenvolvimento de software.",
  "createdAt": "2026-07-01T12:00:00.000Z",
  "updatedAt": "2026-07-01T12:00:00.000Z"
}
```

GET /cursos

Lista os cursos cadastrados. A rota aceita parâmetros de consulta para paginação e filtro textual.

Parametro	Tipo	Descrição
page	number	Número da página desejada.
limit	number	Quantidade máxima de registros por página.
nome	string	Filtro opcional pelo nome do curso.

Exemplo de resposta paginada:

```
{
  "data": [
    {
      "id": 1,
      "nome": "Análise e Desenvolvimento de Sistemas",
      "descricao": "Curso superior de tecnologia focado em desenvolvimento de software.",
      "createdAt": "2026-07-01T12:00:00.000Z",
      "updatedAt": "2026-07-01T12:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  }
}
```

GET /cursos/{id}

Consulta os dados de um curso específico a partir do identificador informado na rota. Pode retornar 200 OK quando o curso existe ou 404 Not Found quando o identificador não corresponde a nenhum registro.

PATCH /cursos/{id}

Atualiza parcialmente os dados de um curso. O cliente pode enviar apenas os campos que deseja modificar. As validações dos campos continuam sendo aplicadas.

```
{
  "descricao": "Curso focado em desenvolvimento, banco de dados e sistemas web."
}
```

DELETE /cursos/{id}

Remove um curso cadastrado. A exclusão pode retornar 200 OK em caso de sucesso ou 404 Not Found quando o curso não for encontrado.

7.3 Alunos

Método	Rota	Descrição
POST	/alunos	Cria um novo aluno vinculado a um curso.
GET	/alunos	Lista alunos com filtros opcionais.
GET	/alunos/{id}	Consulta um aluno específico.
PATCH	/alunos/{id}	Atualiza parcialmente os dados de um aluno.
DELETE	/alunos/{id}	Remove um aluno existente.

POST /alunos

```
{
  "nome": "Maria Oliveira",
  "email": "maria.oliveira@email.com",
  "matricula": "ADS2026001",
  "cursoId": 1
}
```

Campo	Validacoes
nome	Obrigatorio, texto, entre 3 e 100 caracteres.
email	Obrigatorio, e-mail valido.
matricula	Obrigatorio, texto, entre 5 e 20 caracteres.
cursoId	Obrigatorio, inteiro positivo.

Regra de negócio	Descrição
E-mail único	Não deve haver outro aluno com o mesmo e-mail.
Matrícula única	Não deve haver outro aluno com a mesma matrícula.
Curso existente	O curso informado em cursoId deve existir.

Exemplo de resposta 201:

```
{
  "id": 1,
  "nome": "Maria Oliveira",
  "email": "maria.oliveira@email.com",
  "matricula": "ADS2026001",
  "cursoId": 1,
  "createdAt": "2026-07-01T12:00:00.000Z",
  "updatedAt": "2026-07-01T12:00:00.000Z"
}
```

GET /alunos

Lista alunos cadastrados com suporte a paginação e filtros.

Parametro	Tipo	Descrição
page	number	Número da página desejada.
limit	number	Quantidade maxima de registros por página.
nome	string	Filtro opcional pelo nome do aluno.
cursoId	number	Filtro opcional para listar alunos vinculados a um curso específico.

GET /alunos/{id}

Consulta um aluno específico pelo identificador. Retorna 200 OK para registros encontrados ou 404 Not Found quando o aluno não existir.

PATCH /alunos/{id}

Atualiza parcialmente os dados do aluno. Também deve respeitar as regras de e-mail único, matrícula única e curso existente quando esses campos forem enviados.

```
{
  "email": "maria.atualizada@email.com"
}
```

DELETE /alunos/{id}

Remove um aluno cadastrado a partir do seu identificador.

7.4 Disciplinas

Método	Rota	Descrição
POST	/disciplinas	Cria uma nova disciplina.
GET	/disciplinas	Lista disciplinas com filtros.
GET	/disciplinas/{id}	Consulta uma disciplina específica.
GET	/disciplinas/{id}/prerequisitos	Lista os pré-requisitos de uma disciplina.
PATCH	/disciplinas/{id}	Atualiza parcialmente uma disciplina.
DELETE	/disciplinas/{id}	Remove uma disciplina existente.

POST /disciplinas

```
{
  "nome": "Programacao Web",
  "codigo": "WEB101",
  "cargaHoraria": 80,
  "limiteVagas": 40,
  "cursoId": 1,
  "prerequisitoIds": [],
  "ativa": true
}
```

Campo	Validacoes
nome	Obrigatorio, entre 3 e 120 caracteres.
código	Obrigatorio, entre 2 e 20 caracteres.
cargaHoraria	Inteiro positivo, máximo 400.
limiteVagas	Inteiro positivo, máximo 500.
cursoId	Inteiro positivo.
prerequisitoIds	Lista única de inteiros positivos.
ativa	Booleano opcional, padrão true.

Regra de negócio	Descrição
Curso existente	A disciplina deve estar vinculada a um curso valido.
Código único	Não deve haver outra disciplina cadastrada com o mesmo código.
Pré-requisitos existentes	Todos os identificadores informados em prerequisitoIds devem existir.

Regra de negócio	Descrição
Sem auto pré-requisito	Uma disciplina não pode ser pré-requisito de si mesma.
Sem dependência circular	A cadeia de pré-requisitos não pode gerar ciclo entre disciplinas.

Exemplo de resposta 201 com vagas disponíveis:

```
{
  "id": 1,
  "nome": "Programacao Web",
  "codigo": "WEB101",
  "cargaHoraria": 80,
  "limiteVagas": 40,
  "vagasDisponiveis": 40,
  "cursoId": 1,
  "prerequisitoIds": [],
  "ativa": true,
  "createdAt": "2026-07-01T12:00:00.000Z",
  "updatedAt": "2026-07-01T12:00:00.000Z"
}
```

GET /disciplinas

Lista disciplinas cadastradas com filtros opcionais.

Parametro	Tipo	Descrição
page	number	Número da página desejada.
limit	number	Quantidade maxima de registros por página.
nome	string	Filtro opcional pelo nome da disciplina.
cursoId	number	Filtro opcional por curso.
ativa	boolean	Filtro opcional por situação ativa ou inativa.

GET /disciplinas/{id}

Consulta uma disciplina específica a partir do identificador informado.

GET /disciplinas/{id}/prerequisitos

Retorna a lista de disciplinas configuradas como pré-requisito da disciplina informada. Essa rota auxilia na visualização de relacionamentos academicos entre disciplinas.

PATCH /disciplinas/{id}

Atualiza parcialmente uma disciplina. As mesmas validações e regras de negócio aplicadas na criação devem ser respeitadas quando os campos correspondentes forem modificados.

```
{
  "limiteVagas": 45,
  "ativa": true
}
```

DELETE /disciplinas/{id}

Remove uma disciplina existente. Pode retornar erro caso o identificador não seja encontrado ou caso exista alguma regra de relacionamento impedindo a remoção.

7.5 Matrículas

Método	Rota	Descrição
POST	/matriculas	Cria uma matrícula entre aluno e disciplina.
GET	/matriculas	Lista todas as matrículas cadastradas.
GET	/matriculas/{id}	Consulta uma matrícula específica.
GET	/matriculas/alunos/{id}/disciplinas	Lista disciplinas vinculadas a um aluno.
GET	/matriculas/disciplinas/{id}/alunos	Lista alunos vinculados a uma disciplina.
GET	/matriculas/cursos/{id}/alunos	Lista alunos relacionados a um curso.

POST /matriculas

```
{
  "alunoId": 1,
  "disciplinaId": 1
}
```

Campo	Validacoes
alunoId	Obrigatorio, inteiro positivo.
disciplinaId	Obrigatorio, inteiro positivo.

Regra: o mesmo aluno não pode ser matriculado duas vezes na mesma disciplina.

Observação: na implementacao atual, o serviço de matrículas valida duplicidade, mas não valida a existencia do aluno ou da disciplina informados.

Exemplo de resposta 201:

```
{
  "id": 1,
  "alunoId": 1,
  "disciplinaId": 1,
  "createdAt": "2026-07-01T12:00:00.000Z"
}
```

GET /matriculas

Lista todas as matrículas cadastradas no sistema.

```
[
  {
    "id": 1,
    "alunoId": 1,
    "disciplinaId": 1,
    "createdAt": "2026-07-01T12:00:00.000Z"
  }
]
```

GET /matriculas/{id}

Consulta uma matrícula específica pelo identificador.

GET /matriculas/alunos/{id}/disciplinas

Retorna as disciplinas associadas ao aluno informado. Essa rota representa a consulta de relacionamento aluno -> disciplinas usando o prefixo de matrículas.

GET /matriculas/disciplinas/{id}/alunos

Retorna os alunos associados a uma disciplina específica, permitindo visualizar quais alunos estão matriculados em determinada disciplina.

GET /matriculas/cursos/{id}/alunos

Retorna alunos relacionados ao curso informado.

Observação: este endpoint está implementado no controller, mas no service atual retorna lista vazia.

8. Observações sobre rotas de relacionamento

O enunciado do papel de relacionamentos citava rotas diretas a partir dos recursos principais. Entretanto, a implementação atual expõe essas consultas com o prefixo /matriculas.

Rotas citadas no enunciado	Rotas expostas na implementação atual
/alunos/{id}/disciplinas	/matriculas/alunos/{id}/disciplinas
/disciplinas/{id}/alunos	/matriculas/disciplinas/{id}/alunos
/cursos/{id}/alunos	/matriculas/cursos/{id}/alunos

Essa diferença deve ser registrada na documentação para evitar erro de integração. Na prática, o consumidor da API deve usar as rotas efetivamente expostas pelo controller, ou seja, as rotas iniciadas por /matriculas.

9. Fluxo de uso da API

Um fluxo básico de uso da API pode ser executado na seguinte ordem:

Etapa	Ação	Endpoint sugerido
1	Criar curso.	POST /cursos
2	Criar aluno vinculado ao curso.	POST /alunos
3	Criar disciplina vinculada ao curso.	POST /disciplinas
4	Criar matrícula entre aluno e disciplina.	POST /matriculas
5	Consultar relacionamentos.	GET /matriculas/alunos/{id}/disciplinas ou rotas equivalentes

Essa sequência respeita as dependências naturais entre os recursos. Primeiro é necessário existir um curso, depois um aluno e uma disciplina relacionados a esse curso, e por fim uma matrícula conectando aluno e disciplina.

10. Conclusão

A documentação OpenAPI do projeto Gerenciador de Alunos centraliza os contratos da API e reduz erros de integração entre backend e front-end. Ao apresentar rotas, parâmetros, exemplos, status codes, casos de erro e observações de implementação, o documento se torna uma referência única para desenvolvimento, testes, manutenção e avaliação acadêmica.

O Swagger UI complementa essa documentação ao transformá-la em uma interface interativa. Com ele, os integrantes do grupo conseguem visualizar endpoints, conferir schemas e testar chamadas da API durante o desenvolvimento, facilitando a apresentação técnica e a validação dos requisitos do trabalho.